

Sistema de Reserva de Canchas Deportivas

Microfrontends con Module Federation y microservicios Spring Boot

Pablo Jara Muñoz · Miguel Lara · Sebastián Uyaguari · Adrián Espinoza

[Ciudad], 25 de agosto de 2026

Maestría en Ingeniería de Software · Desarrollo de Aplicaciones Empresariales

[Nombre de la Universidad]

Qué vamos a ver

Guion. Recorrer la agenda en veinte segundos, apoyándose en que la demo está en el centro y no al final: es lo que compra atención durante los bloques de arquitectura.

1. El problema y el alcance acordado
2. Cómo lo construimos: especificación primero
3. Arquitectura de la solución
4. Frontend: Module Federation
5. Backend: microservicios Spring Boot
6. Modelo de datos
7. Reglas de negocio
8. Reportes
9. Demostración en vivo
10. Calidad y despliegue
11. Cierre y preguntas

Duración prevista: 20 minutos + 5 de preguntas.

El problema y el alcance

El problema y el alcance

Duración: 1,5 min • **Expone:** [Integrante]

Criterio de la rúbrica: Alcance funcional implementado (25 %)

Guion. *Una frase de contexto y pasar. El docente escribió el documento de alcance: no hay que explicárselo, hay que demostrar que se leyó.*

Un club con canchas de **pádel, tenis y básquet** necesita que:

- el socio vea qué bloques quedan libres, reserve y cancele;
- el administrador gestione el catálogo, cancele ante un mantenimiento imprevisto y mida el uso real.

Dos roles, cuatro módulos, ocho reglas de negocio.

Lo que NO está en alcance

Pagos en línea · notificaciones · reservas recurrentes · torneos · app móvil nativa · BI avanzado.

Delimitado así en el documento de alcance, §3.5. No lo construimos, y es deliberado.

Las cuatro historias que sostienen el alcance

Usuario final

Consultar disponibilidad por cancha y fecha

Reservar un bloque libre

Ver sus reservas y cancelarlas

Administrador

Alta, edición e inactivación de canchas

Bloqueos de mantenimiento

Cancelar cualquier reserva

Activar e inactivar usuarios

Los cuatro indicadores de uso

Las cuatro están construidas y son demostrables por separado. [Se verán en vivo en el bloque 9.](#)

Cómo lo construimos

Duración: 1,5 min • **Expone:** [Integrante]

Criterio de la rúbrica: Calidad técnica y documentación (10 %)

Especificación primero, no «vibe coding»

Guion. Enlazar explícitamente con la sesión del curso. El mensaje: elegimos el enfoque que el propio curso señala como el indicado para sistemas con reglas de negocio y varios equipos tocando el mismo contrato.

Vibe coding

Se describe el destino, no la ruta. Fricción mínima, iteración conversacional.

Bien para: prototipos, exploración de una API, demos.

Spec-Driven Development

La especificación es la fuente de verdad; el código es su consecuencia.

Necesario cuando hay: reglas de negocio críticas, contratos entre equipos, trazabilidad exigible.

Este proyecto tiene las tres cosas. Elegimos la segunda.

El flujo, y lo que dejó como evidencia

Guion. Enseñar el camino de un requisito hasta su prueba. Es el argumento que sostiene la tabla de reglas de negocio del bloque 7: cada RN apunta a una clase y a un test con nombre propio.

constitución → **spec** → plan → tasks → implementación → pruebas

- Contrato OpenAPI acordado *antes* de escribir el servicio.
- Criterios de aceptación como pruebas, no como prosa.
- Cada regla de negocio tiene una clase y un test con nombre.
- Las compuertas de revisión quedan en el historial del repositorio.

Lo que compró

Poder responder «¿de dónde sale esto?» señalando un archivo, en vez de reconstruirlo de memoria.

Arquitectura de la solución

Duración: 2 min · **Expone:** [Integrante]

Criterio de la rúbrica: Base de los criterios de microfrontends y microservicios (30 %)

Guion. *Veinte segundos. Solo para que nadie pregunte «¿qué notación es esta?» en mitad del diagrama siguiente.*

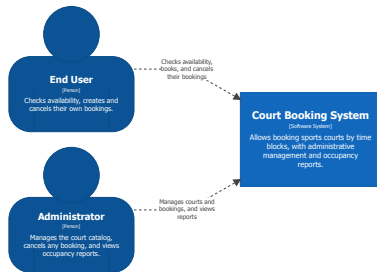
Modelo **C4**, cuatro vistas, generadas desde una única descripción en Structurizr DSL:

- V-01 Contexto — quién usa el sistema
- V-02 Contenedores — las once piezas que se despliegan
- V-03 Componentes de **ms-reservas**
- V-04 Despliegue — cómo caen en Docker

Por qué importa

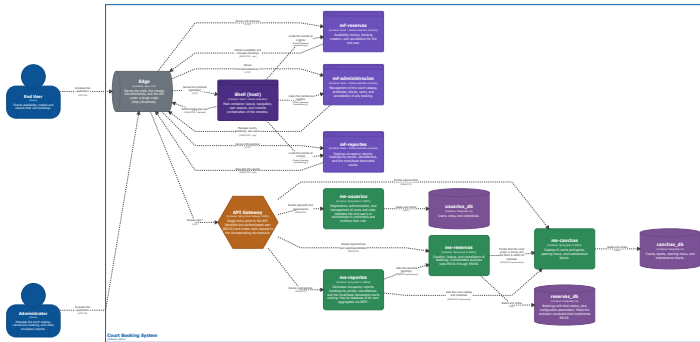
Un solo origen para los diagramas: no hay una versión en el informe y otra distinta en las diapositivas.

Contexto del sistema (V-01)



Socio y administrador frente al sistema. Sin integraciones externas: es parte del alcance acordado.

Contenedores (V-02)



Once contenedores. El **edge** es el único conectado a las dos redes: es lo que impide alcanzar un microservicio saltándose el gateway.

Tres decisiones que explican el resto de la charla

ADR-01 — Solapamiento resuelto en la base de datos

Decisión. Restricción **EXCLUDE** sobre **tstzrange** en **reservas_db**, además de la comprobación en el servicio.

Descartado. Confiar solo en la validación aplicativa: bajo peticiones concurrentes deja pasar la doble reserva.

ADR-02 — Una base por microservicio, con rol propio

Decisión. Tres bases aisladas por credenciales; ningún servicio lee las tablas de otro aunque quiera.

Descartado. Esquema compartido: convierte la independencia de datos en una promesa que nadie verifica.

ADR-03 — Un edge que unifica el origen

Decisión. Nginx sirve shell, remotes y **/api** bajo un mismo origen.

Descartado. CORS servicio por servicio: filtra los puertos internos al navegador.

Frontend: Module Federation

Duración: 2 min · **Expone:** [Integrante]

Criterio de la rúbrica: Arquitectura de microfrontends (15 %)

Un host y tres remotes

Guion. *Nombrar los cuatro y su responsabilidad. No entrar todavía en cómo se cargan: eso es la lámina siguiente.*

Microfrontend	Tipo	Responsabilidad
shell	Host	Layout, navegación, sesión y orquestación de los remotes
mf-reservas	Remote	Disponibilidad, nueva reserva, mis reservas
mf-administracion	Remote	Canchas, bloqueos, reservas y usuarios
mf-reportes	Remote	Los cuatro indicadores de uso

La rúbrica pide un host y *al menos dos* remotes. [Entregamos tres](#), cada uno con su `package.json`, su build y su imagen.

[Fragmento real de rsbuild.config.ts del shell: bloque `moduleFederation` con `remotes` y `shared`]

Recórtalo a diez líneas. En pantalla, un archivo entero no se lee.

- Los remotes se cargan por URL, en ejecución.
- **shared**: React y el enrutador se cargan una sola vez.
- El shell inyecta la sesión; el remote no la reimplementa.
- Si un remote no responde, el shell degrada esa ruta y sigue en pie.

La prueba de que son independientes

Guion. Esta es la lámina que casi nadie trae. Si hay tiempo en la demo, hacerlo en vivo: reconstruir un remote con el shell corriendo y recargar.

Demostración de despliegue independiente

1. El sistema está corriendo, con el shell servido por el **edge**.
2. Se cambia algo visible en **mf-reportes** y se reconstruye solo ese contenedor.
3. Se recarga el navegador: el cambio está. El shell y los otros dos remotes nunca se reconstruyeron.

[Captura o comando exacto que se usará: `docker compose build/up` de un solo servicio]

Backend: microservicios Spring Boot

Duración: 2 min · **Expone:** [Integrante]

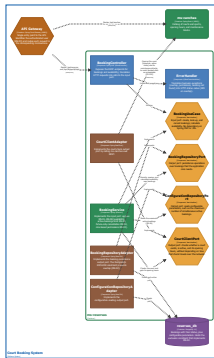
Criterio de la rúbrica: Arquitectura de microservicios (15 %)

Cuatro servicios y un gateway

Servicio	Base	Responsabilidad
<code>api-gateway</code>	—	Entrada única; verifica la credencial y rutea
<code>ms-usuarios</code>	<code>usuarios_db</code>	Registro, autenticación, usuarios y roles
<code>ms-canchas</code>	<code>canchas_db</code>	Catálogo, horarios de atención y bloqueos
<code>ms-reservas</code>	<code>reservas_db</code>	Disponibilidad, creación y cancelación
<code>ms-reportes</code>	<code>sin base</code>	Indicadores, componiendo datos de los otros dos

`ms-reportes` no persiste nada: compone por HTTP lo que ya está en `ms-reservas` y `ms-canchas`.

Componentes de ms-reservas (V-03)



El servicio donde viven las reglas de negocio. Adaptadores fuera, dominio dentro: el dominio no conoce ni HTTP ni JPA.

Guion. *Aquí se abre Swagger si el proyector coopera; si no, la captura. El mensaje: el contrato se acordó primero y el servicio se escribió contra él, no al revés.*

- Cada servicio publica su OpenAPI en `/swagger-ui`.
- El gateway propaga identidad y rol en cada petición (**X-User-Role**); el servicio no vuelve a confiar en el navegador.
- Comunicación síncrona por HTTP entre servicios, sin cola de mensajes: el alcance no la justifica.
- Colección de pruebas de la API como parte del entregable **E3**.

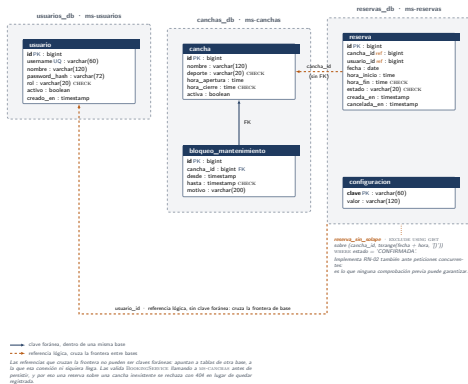
[Captura de Swagger UI de ms-reservas, recortada a los cuatro endpoints principales]

Modelo de datos

Duración: 1 min • **Expone:** [Integrante]

Criterio de la rúbrica: Modelo de datos y persistencia (15 %)

Modelo entidad-relación



Tres bases separadas, no tres esquemas del mismo modelo. Las líneas que cruzan la frontera son referencias por identificador, no claves foráneas.

La independencia de datos es verificable

Guion. *El argumento fuerte: no es una convención de equipo, es un permiso denegado por PostgreSQL. Si alguien duda, se puede probar en la demo.*

- Un rol de PostgreSQL por servicio, con permisos solo sobre su base.
- **ms-reservas** guarda el **id** de la cancha, no una clave foránea a **canchas_db**: la coherencia se valida llamando al servicio dueño del dato.
- El esquema y los datos de prueba se crean solos al levantar (**E4**): ningún paso manual.

Cómo se comprueba

Conectarse con el rol de **ms-reservas** e intentar leer una tabla de **canchas_db**: **permiso denegado**.

[Comando psql exacto, ya probado, por si lo piden]

Reglas de negocio

Duración: 2 min · **Expone:** [Integrante]

Criterio de la rúbrica: Reglas de negocio y validaciones (10 %)

Ocho reglas, ocho lugares donde vive cada una

Guion. No leer la tabla entera. Señalar la columna de la derecha y decir la idea: ninguna regla vive solo en la prosa del informe.

Regla	Dónde vive
RN-01 Cancha, fecha y bloque de una hora	<code>BookingService.crear · TimeSlot</code>
RN-02 Sin solapamiento, ni bajo concurrencia	Servicio <code>y EXCLUDE</code> en la base
RN-03 Cada quien cancela lo suyo; el admin, todo	<code>BookingService.cancelar</code>
RN-04 No se cancela una reserva ya empezada	<code>Booking.sePuedeCancelarEn</code>
RN-05 Cancelar libera el bloque, sin borrar	<code>Booking.cancelar</code>
RN-06 Tope de reservas activas (3 por defecto)	<code>BookingService.verificarTope</code>
RN-07 Solo el admin gestiona el catálogo	<code>CourtService</code> en <code>ms-canchas</code>
RN-08 Confirmada / Cancelada / Finalizada	<code>Booking.estadoVigente</code>

Cada fila tiene además una prueba con nombre propio. La correspondencia completa está en el informe (E1, §6).

RN-02: por qué no basta con validar en el servicio

Guion. *La lámina técnica de la charla. Contar la carrera: dos peticiones simultáneas, las dos preguntan «¿está libre?», las dos oyen que sí. La comprobación en el servicio es necesaria para dar un mensaje decente; la restricción es la que hace que la regla sea cierta.*

El problema

Dos peticiones concurrentes sobre el mismo bloque consultan antes de que ninguna haya escrito. Ambas ven el bloque libre. Ambas insertan.

La solución

La regla la impone PostgreSQL. Si la carrera ocurre, la segunda inserción falla y el servicio la traduce a un **409 Conflict**.

[Fragmento del DDL con la restricción EXCLUDE sobre tstzrange, recortado a cinco o seis líneas]

Y la prueba que lo respalda:

ConcurrenciaReservaIT, que lanza las dos peticiones a la vez y exige que exactamente una gane.

Reportes

Duración: 1 min • **Expone:** [Integrante]

Criterio de la rúbrica: Módulo de reportes básicos (10 %)

Cuatro indicadores, sobre el rango que elija el administrador

Guion. Enumerar los cuatro rápido y anunciar la prueba de consistencia: en la demo se crea una reserva y se vuelve aquí a ver el número subir. Es lo que separa un reporte de una pantalla con datos de adorno.

1. Reservas por cancha y por deporte, en un rango de fechas.
2. Porcentaje de ocupación por cancha (horas reservadas sobre horas disponibles).
3. Cancelaciones por período.
4. Canchas de mayor y menor demanda.

[Captura del panel de reportes con datos de prueba]

Solo lectura, solo administrador. Sin almacén analítico:

ms-reportes compone lo que ya está en

ms-reservas y **ms-canchas**.

Demostración en vivo

Duración: 4 min • **Expone:** [Integrante]

Criterio de la rúbrica: Alcance funcional implementado (25 %)

Guion. Esta lámina se proyecta mientras se cambia de ventana: le dice al público qué va a ver y evita el silencio incómodo. Se sale de ella en cuanto el navegador está delante.

Como socio

1. Entrar y ver la disponibilidad de una cancha para hoy.
2. Reservar un bloque libre.
3. Intentar reservar **el mismo bloque** desde otra sesión → el sistema lo rechaza (RN-02).
4. Ver «mis reservas» y cancelar una.
5. Comprobar que el bloque volvió a estar libre (RN-05).

Como administrador

6. Crear una cancha y verla aparecer en el catálogo del socio.
7. Cancelar la reserva de otro usuario (RN-03).
8. Abrir reportes: el número refleja lo que acaba de pasar.

Si sobra tiempo

Reconstruir un solo remote con el sistema en pie.

- `docker compose up -d --wait` lanzado **antes** de entrar al aula; `docker compose ps` con los once servicios en **healthy**.
- Dos navegadores (o uno normal y uno privado) ya con sesión iniciada: uno como socio, otro como administrador.
- Datos de prueba recién sembrados: sin reservas viejas que ensucien los reportes.
- Zoom del navegador al **[120 %]**: lo que se lee en el portátil no se lee en el proyector.
- Notificaciones del sistema operativo silenciadas.
- **Plan B:** un vídeo corto de la demo, ya en el escritorio. **[Grabarlo]**

Calidad y despliegue

Duración: 1 min • **Expone:** [Integrante]

Criterio de la rúbrica: Calidad técnica y documentación (10 %)

1

comando para levantar todo

- No hace falta Java, Node, Maven ni PostgreSQL en la máquina: todo se compila dentro de los contenedores.
- Esquema y datos de prueba se crean al arrancar.
- Puertos sustituibles por variable de entorno, por si el evaluador ya tiene el 8080 ocupado.
- Manual de despliegue como entregable aparte (E5).

11

contenedores

1

requisito: Docker

Por qué es una decisión, no comodidad

Que la evaluación no dependa de qué tenga instalado quien evalúa.

Qué atributos de calidad perseguimos, y cómo

Atributo	Mecanismo concreto en este sistema
Mantenibilidad	Dominio aislado de HTTP y de JPA; cada regla con su prueba
Escalabilidad	Servicios sin estado tras el gateway; se replican por separado
Seguridad	Bases aisladas por credenciales; el navegador nunca alcanza un servicio directamente
Disponibilidad	healthcheck por contenedor; un remote caído degrada una ruta, no la aplicación

[Añade aquí la cifra de cobertura de pruebas o el número de pruebas que corren, si la tienes]

Cierre

Duración: 1,5 min · **Expone:** [Integrante]

Criterio de la rúbrica: Todos

Los seis criterios de aceptación

Criterio del alcance (§7)	Evidencia
✓ CA-1 Socio consulta, reserva y cancela	Demo, pasos 1–5
✓ CA-2 Admin gestiona canchas y cancela cualquier reserva	Demo, pasos 6–7
✓ CA-3 No se reserva un bloque ocupado	Demo, paso 3; RN-02 y ConcurrenciaReservaIT
✓ CA-4 Reportes de ocupación y reservas por período	Demo, paso 8; cuatro indicadores
✓ CA-5 Shell y ≥ 2 remotes con Module Federation	Tres remotes; prueba de despliegue independiente
✓ CA-6 ≥ 2 microservicios en PostgreSQL	Cuatro servicios, tres bases aisladas

Lo que aprendimos y lo que dejamos abierto

Guion. *Honestidad concreta, no humildad genérica. «Trabajo futuro» que no sea la lista de cosas fuera de alcance: eso no es aprendizaje, es el enunciado.*

Lecciones

- Una regla crítica que solo vive en el código de aplicación no es una regla: es una intención. **RN-02** nos lo enseñó.
- La independencia de datos hay que hacerla imposible de violar, no acordarla en una reunión.
- Escribir el contrato antes ahorra más tiempo del que costó.
- [La tercera, la vuestra de verdad]

Trabajo futuro

- [Lo que quedó a medias y sabéis cómo terminar]
- [Lo que la arquitectura ya permite y no construimos]

Gracias

¿Preguntas?

Guion. *Para el ponente. Apunta el número de lámina de cada fila cuando la presentación esté congelada.*

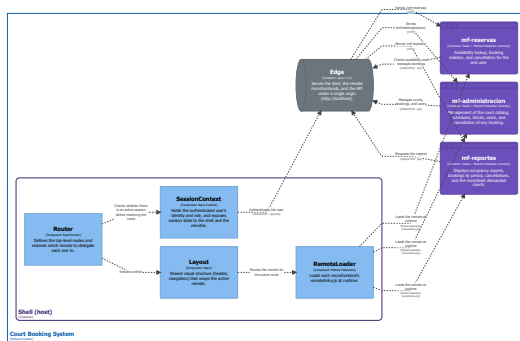
- Vistas C4 restantes: componentes de cada servicio y microfrontend
- Despliegue (V-04) y el papel del **edge**
- Registro completo de decisiones (ADR)
- Diccionario de datos y DDL
- Traducción de reglas de negocio a códigos HTTP
- Estrategia de pruebas
- Por qué DDD y dónde están los agregados

Despliegue (V-04)



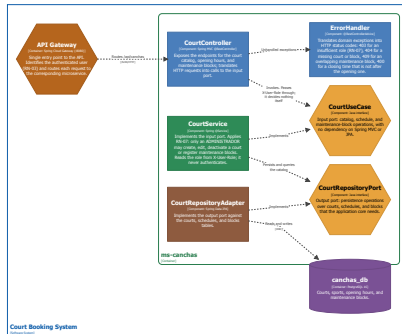
Dos redes Docker. Solo el **edge** toca las dos.

Componentes del shell

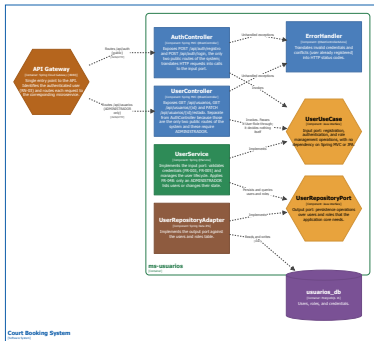


Cómo el host resuelve los remotes y comparte la sesión.

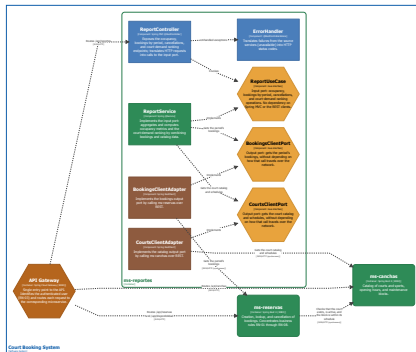
Componentes de ms-canchas



Componentes de ms-usuarios

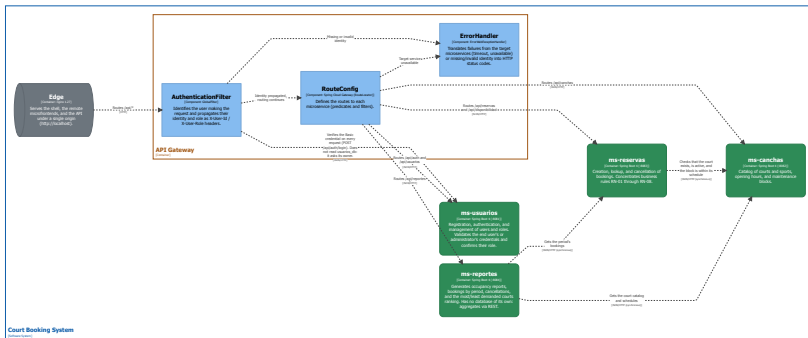


Componentes de ms-reportes



El único servicio sin base de datos: compone por HTTP.

Componentes del api-gateway



Dónde se verifica la credencial antes de rutear.

Reglas de negocio → respuestas HTTP

Guion. Respuesta preparada para «¿y qué le devuelve al cliente cuando la regla no se cumple?». Rellenar con lo que hace el servicio de verdad.

Situación	Código	Regla
Bloque ya ocupado	409	RN-02
Cancelar una reserva ajena	403	RN-03
Cancelar una reserva pasada	422	RN-04
Tope de reservas alcanzado	422	RN-06
[Completar con el resto]	[—]	

Verifica cada código contra el controlador antes de exponer: un número inventado aquí se nota enseguida.

[Pirámide de pruebas del proyecto: unitarias de dominio, de integración con base real, y de contrato. Cifras concretas si las tenéis.]

Preguntas que esperamos

Guion. *Ensayad las respuestas en voz alta. Una respuesta pensada en el momento suena a que no se pensó nunca.*

- ¿Por qué microservicios para un sistema de este tamaño?
- ¿Qué pasa si **ms-canchas** está caído cuando alguien reserva?
- ¿Por qué no una cola de mensajes entre servicios?
- ¿Cómo escalaríais si mañana hubiera cien clubes?
- ¿Module Federation no acopla los remotes al shell por la versión de React?
- ¿Cómo evitáis que un remote roto tumbe toda la aplicación?
- [La que os dé más miedo]